MATERIAL DIDÁTICO DO PROFESSOR • AULA 8

Profissionalizando o Tratamento de Exceções e Refatoração Arquitetural

Adequando o Invest-Tracker aos Padrões Corporativos: Fim dos try/catch nos Controllers, Criação de Exceções Semânticas, DTOs Imutáveis com Records e Interceptação Global com @ServerExceptionMapper.

☕ Java 21 | ⚡ Quarkus 3.39+ | 🛡️ GlobalExceptionHandler | 📦 Records & DTOs | 💾 Panache Repositories | 🎯 Clean Architecture

1 O Diagnóstico: Por que o Código Anterior Estava Frágil?

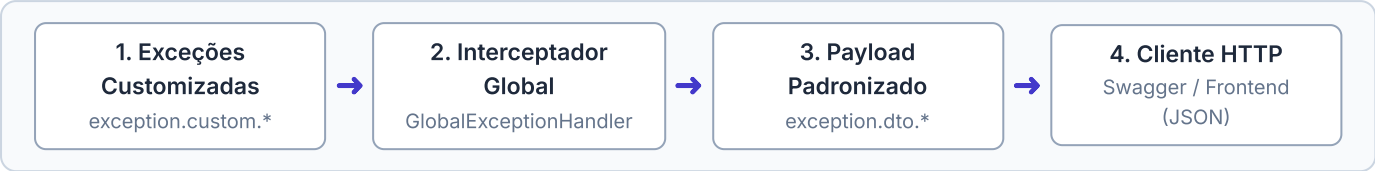
Ao analisar a versão inicial do projeto **Invest-Tracker**, identificamos quatro grandes problemas arquiteturais que impediam a aplicação de compilar e seguir o padrão de mercado:

Problema Encontrado	Impacto no Sistema	Solução Corporativa Aplicada
Blocos try/catch genéricos nos Resources	Todo erro (inclusive 404 Not Found e 409 Conflict) era capturado e transformado em `500 Internal Server Error`, com mensagens soltas em texto puro.	**Remoção total dos try/catch** nos Resources. Delegação automática para o `GlobalExceptionHandler`.
Anotações inválidas do Bean Validation	Uso de `@NotBlank` em atributos como `LocalDateTime`, `BigDecimal` e objetos de Entidade (que só funcionam com Strings), gerando erros de runtime.	Uso correto de `@NotNull`, `@Positive`, `@PositiveOrZero` e isolamento com `UUID` nos Forms de entrada.
Ausência de @ApplicationScoped nos Services	Os Services não estavam registrados no contexto CDI do Arc/Quarkus, impossibilitando a injeção via `@Inject`.	Anotação `@ApplicationScoped` adicionada a todos os 6 Services e Mappers da aplicação.
Dependência de ModelMapper inexistente no POM	Classes de mapeamento tentavam dar `new ModelMapper()` sem a biblioteca no `pom.xml`, quebrando a compilação do Maven.	Mapeamento direto, limpo e performático usando construtores e **Java 21 Records** (sem reflection, nativo para GraalVM).

⚠️ **O Anti-Pattern do Resource Poluído:** Quando um Controller tenta fazer o papel de interceptador de erros com blocos `try/catch` repetitivos, ele esconde a causa raiz dos problemas, viola o princípio de responsabilidade única (SRP) e impede que o cliente HTTP receba o status correto de resposta.

2 A Nova Arquitetura de Tratamento de Exceções

Implementamos o padrão em **3 camadas** sob o pacote `backend/src/main/java/exception/` :



2.1. Camada de Exceções Semânticas (custom/)

Criamos exceções não-checadas (`RuntimeException`) com claro significado semântico:

1. ResourceNotFoundException (HTTP 404): Lançada quando o registro solicitado por ID não existe.

```
package exception.custom;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

2. ConflictException (HTTP 409): Lançada em caso de violação de chaves únicas ou exclusão indevida de dados vinculados.

```
package exception.custom;

public class ConflictException extends RuntimeException {
    public ConflictException(String message) {
        super(message);
    }
}
```

3. BusinessException (HTTP 400): Lançada quando regras de negócio são violadas (ex: tipo de operação inválido).

```
package exception.custom;

public class BusinessException extends RuntimeException {
    public BusinessException(String message) {
        super(message);
    }
}
```

2.2. Camada de Contrato JSON (dto/)

Toda resposta de erro da API agora segue rigorosamente a mesma estrutura contratual:

ErrorResponseDTO.java:

```java
package exception.dto;

import java.time.LocalDateTime;
import java.util.List;

public record ErrorResponseDTO(
    int status,
    String error,
    String message,
    String path,
    LocalDateTime timestamp,
    List<ValidationErrorDTO> fieldErrors
) {
    public ErrorResponseDTO(int status, String error, String message, String path) {
        this(status, error, message, path, LocalDateTime.now(), null);
    }
}
```

ValidationErrorDTO.java:

```java
package exception.dto;

public record ValidationErrorDTO(
    String field,
    String message
) {}
```

2.3. O Interceptador Global (handler/GlobalExceptionHandler.java)

No Quarkus REST, utilizamos o recurso reativo `@ServerExceptionMapper`. Quando qualquer Service ou validador dispara uma exceção, ela é capturada antes de sair do servidor:

```java
package exception.handler;

import exception.custom.*;
import exception.dto.*;
import jakarta.validation.ConstraintViolationException;
import jakarta.ws.rs.core.Response;
import jakarta.ws.rs.core.UriInfo;
import org.jboss.resteasy.reactive.server.ServerExceptionMapper;

public class GlobalExceptionHandler {

    @ServerExceptionMapper
    public Response handleNotFound(ResourceNotFoundException ex, UriInfo uriInfo) {
        var error = new ErrorResponseDTO(404, "Not Found", ex.getMessage(), uriInfo.getPath()
        return Response.status(Response.Status.NOT_FOUND).entity(error).build();
    }

    @ServerExceptionMapper
    public Response handleConflict(ConflictException ex, UriInfo uriInfo) {
        var error = new ErrorResponseDTO(409, "Conflict", ex.getMessage(), uriInfo.getPath())
        return Response.status(Response.Status.CONFLICT).entity(error).build();
    }

    @ServerExceptionMapper
    public Response handleBusiness(BusinessException ex, UriInfo uriInfo) {
        var error = new ErrorResponseDTO(400, "Bad Request", ex.getMessage(), uriInfo.getPath
        return Response.status(Response.Status.BAD_REQUEST).entity(error).build();
    }

    @ServerExceptionMapper
    public Response handleValidation(ConstraintViolationException ex, UriInfo uriInfo) {
        var fieldErrors = ex.getConstraintViolations().stream()
            .map(cv -> new ValidationErrorDTO(
                cv.getPropertyPath().toString().substring(cv.getPropertyPath().toString().las
                cv.getMessage()))
            .toList();

        var error = new ErrorResponseDTO(400, "Validation Error", "Erro de validação",
                                         uriInfo.getPath(), java.time.LocalDateTime.now(), fie
        return Response.status(Response.Status.BAD_REQUEST).entity(error).build();
    }

    @ServerExceptionMapper
    public Response handleGeneric(Throwable ex, UriInfo uriInfo) {
        var error = new ErrorResponseDTO(500, "Internal Server Error", ex.getMessage(), uriIn
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR).entity(error).build();
    }
}
```

2.4. Dúvida Crucial: Se o persist() falhar no banco, quem dá o throw? O Papel do Hibernate e do Throwable

> 💡 **Dúvida Frequente de Aula:** *"No método `AssetService.create()`, após a linha `assetRepository.persist(asset);` não existe nenhum `throw`. Se der erro no banco, como isso vai cair no `GlobalExceptionHandler`?"*

Para entender esse comportamento, precisamos separar em dois caminhos de execução:

1. O Caminho Feliz (Sucesso)

Se o PostgreSQL processar o registro normalmente, **não há throw em lugar nenhum!** E esse é o fluxo normal. A execução passa pelo `persist()`, atinge a linha seguinte (`return assetMapper.toDTO(asset);`), o Quarkus confirma o commit da transação `@Transactional` e o Controller responde **HTTP 201 Created** com o DTO recém-criado.

2. O Caminho de Falha de Infraestrutura (Quem dispara o throw?)

Se o banco de dados cair, houver timeout de rede, estouro de caracteres na coluna SQL ou falha de chave estrangeira no banco, **o desenvolvedor não precisa escrever throw manualmente**. O próprio ecossistema do **Hibernate, Panache e Driver JDBC** dispara internamente uma exceção de runtime (como `PersistenceException`, `SQLException` ou `DataException`).

> 🛡️ **Por que usamos `Throwable` no método genérico?**
> No Java, `Throwable` é a classe-mãe absoluta de todas as exceções e erros da JVM (`Throwable` → `Exception` → `RuntimeException`). Ao declarar `handleGeneric(Throwable ex)` com `@ServerExceptionMapper`, criamos uma **rede de segurança universal (fallback)**. Qualquer erro inesperado que o Hibernate ou o banco disparar e que não tenha um handler específico cairá automaticamente aqui, sendo convertido em um JSON limpo com status **500 Internal Server Error**, sem expor stacktraces sensíveis para o usuário final.

3 Refatoração Prática: Como Ficaram os Recursos e Serviços

Todos os **6 módulos** da aplicação foram refatorados para tirar proveito da nova arquitetura:

3.1. O Novo Padrão de Resource (Zero try/catch)

Observe a elegância do novo `AssetResource.java`. Não há nenhum bloco `try/catch`. Se o serviço falhar, o Quarkus sabe exatamente o que fazer:

```java
@Path("/assets")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class AssetResource {

    @Inject AssetService assetService;

    @GET
    public Response findAll() {
        var assets = assetService.getAllAssets();
        return assets.isEmpty() ? Response.noContent().build() : Response.ok(assets).build();
    }

    @GET @Path("/{id}")
    public Response findById(@PathParam("id") UUID id) {
        return Response.ok(assetService.getById(id)).build();
    }

    @POST
    public Response create(@Valid AssetForm assetForm) {
        return Response.status(Response.Status.CREATED).entity(assetService.create(assetForm)
    }

    @PUT @Path("/{id}")
    public Response update(@PathParam("id") UUID id, @Valid AssetForm assetForm) {
        return Response.ok(assetService.update(id, assetForm)).build();
    }

    @DELETE @Path("/{id}")
    public Response delete(@PathParam("id") UUID id) {
        assetService.delete(id);
        return Response.noContent().build();
    }
}
```

3.2. O Novo Padrão de Service (Protegido e Transacional)

No `AssetService.java`, implementamos regras rigorosas antes de permitir exclusões ou inserções duplicadas:

```java
@ApplicationScoped
public class AssetService {

    @Inject AssetRepository assetRepository;
    @Inject TransactionRepository transactionRepository;
    @Inject DailyQuoteRepository dailyQuoteRepository;
    @Inject AssetMapper assetMapper;

    public AssetDTO getById(UUID id) {
        Asset asset = assetRepository.findByIdOptional(id)
            .orElseThrow(() -> new ResourceNotFoundException("Asset not found for ID: " + id)
        return assetMapper.toDTO(asset);
    }

    @Transactional
    public AssetDTO create(AssetForm form) {
        assetRepository.findByTicker(form.ticker()).ifPresent(existing -> {
            throw new ConflictException("Asset with ticker '" + form.ticker() + "' already ex
        });
        Asset asset = assetMapper.formToModel(form);
        assetRepository.persist(asset);
        return assetMapper.toDTO(asset);
    }

    @Transactional
    public void delete(UUID id) {
        Asset asset = assetRepository.findByIdOptional(id)
            .orElseThrow(() -> new ResourceNotFoundException("Asset not found for ID: " + id)

        if (!transactionRepository.listByAssetId(id).isEmpty()) {
            throw new ConflictException("Not possible to delete asset with registered transac
        }
        if (!dailyQuoteRepository.listByAssetId(id).isEmpty()) {
            throw new ConflictException("Not possible to delete asset with registered daily c
        }
        assetRepository.delete(asset);
    }
}
```

4 O Resultado na Prática: Payloads de Erro Recebidos pelo Cliente

Veja como o cliente da API (Frontend, Swagger UI ou Postman) recebe respostas claras, rastreáveis e semânticas:

Cenário A: Busca por ID inexistente (HTTP 404 Not Found)

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Asset not found for ID: 9c8e1e7f-71b3-4f9e-a0e2-b7b51bfa70d1",
  "path": "/assets/9c8e1e7f-71b3-4f9e-a0e2-b7b51bfa70d1",
  "timestamp": "2026-11-04T17:16:00",
  "fieldErrors": null
}
```

Cenário B: Falha na validação de campos obrigatórios (HTTP 400 Bad Request)

```
{
  "status": 400,
  "error": "Validation Error",
  "message": "Validation error on submitted fields",
  "path": "/users",
  "timestamp": "2026-11-04T17:16:00",
  "fieldErrors": [
    {
      "field": "email",
      "message": "Invalid email format."
    },
    {
      "field": "password",
      "message": "Password must have at least 6 characters."
    }
  ]
}
```

Cenário C: Violação de integridade ou exclusão bloqueada (HTTP 409 Conflict)

```
{
  "status": 409,
  "error": "Conflict",
  "message": "Not possible to delete asset with registered transactions.",
  "path": "/assets/8d7c2a11-5463-4411-9a99-b132890ef011",
  "timestamp": "2026-11-04T17:16:00",
  "fieldErrors": null
}
```

5 Checklist de Boas Práticas Enterprise

✅ **Status Final da Compilação:** Executado `./mvnw compile -DskipTests` com sucesso absoluto (`BUILD SUCCESS`). Todos os 48 arquivos-fonte compilados com Java 21 sem warnings impeditivos.

Regra	Diretriz de Mercado	Status no Invest-Tracker
Camadas	Resource não faz lógica nem acesso a banco; apenas valida HTTP e delega ao Service.	✔ 100% Implementado
Tratamento de Erros	Zero try/catch nos Controllers; captura centralizada com @ServerExceptionMapper.	✔ 100% Implementado
Imutabilidade	Uso de Java 21 Records para DTOs e Forms; isolamento total das Entidades JPA.	✔ 100% Implementado
Validação	Bean Validation (@Valid, @NotNull, @Positive) com mensagens claras de erro.	✔ 100% Implementado
Transacionalidade	Métodos de escrita anotados com @Transactional para garantir rollback automático.	✔ 100% Implementado